MICROBIT. УРОК №2. КОСМИЧЕСКАЯ УГРОЗА.

В 1978 году в Японии произошла странная вещь: в стране закончились монетки номиналом 100 иен. Банкам пришлось чеканить их в экстренном режиме.

Виновником стала игра **Space Invaders** *(рис. 1)*. Она была настолько популярной, что люди выстраивались в очереди и тратили все деньги, чтобы спасти Землю от пиксельных пришельцев.

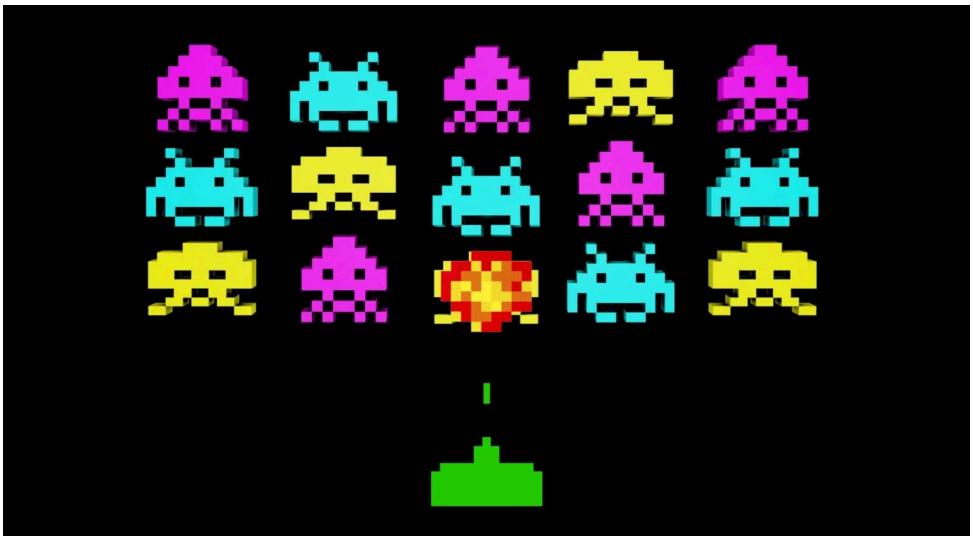


Рисунок 1

Сегодня мы с вами станем создателями этой легенды. Мы напишем свою версию «Захватчиков» для нашей карманной консоли.

На прошлом уроке мы научились опрашивать джойстик. Сегодня мы заставим эти данные

управлять точкой на экране. Но, прежде чем приступать к коду, давайте проверим, как хорошо вы помните архитектуру нашего устройства.

Блиц-опрос!

- **Наш геймпад имеет кнопки и джойстик. Какое из этих устройств выдает цифровой сигнал, а какое аналоговый?**
- **Если мы не трогаем джойстик, какое примерно число он отправляет в Micro:bit?**
- **Зачем нам нужен файл gamepad.py, который мы копировали на прошлом уроке?**
- **Сколько светодиодов на экране Micro:bit?**
- **Когда мы пишем команду if gamepad.button_A():, какой ответ мы ждем от драйвера, если кнопка нажата?**

Экран Micro:bit — это сетка из светодиодов размером 5 на 5 *(рис. 2)*. Чтобы управлять игрой, компьютер должен знать точный адрес каждой точки. Этот адрес называется **координата (X, Y)**.

Справка: Что такое координаты?

Координата *— это точный адрес точки на плоскости.*

Представьте, что экран — это город с улицами.

- *X — это номер улицы (по горизонтали).*
- *Y — это номер дома (по вертикали).*

Зная X и Y, мы можем найти любой светодиод, как почтальон находит нужную квартиру.

- **Ось X (Горизонталь):** Идет слева направо.
  - 0 — левый край.
  - 4 — правый край.
- **Ось Y (Вертикаль):** Идет сверху вниз.
  - 0 — верх.

- 4 — низ.

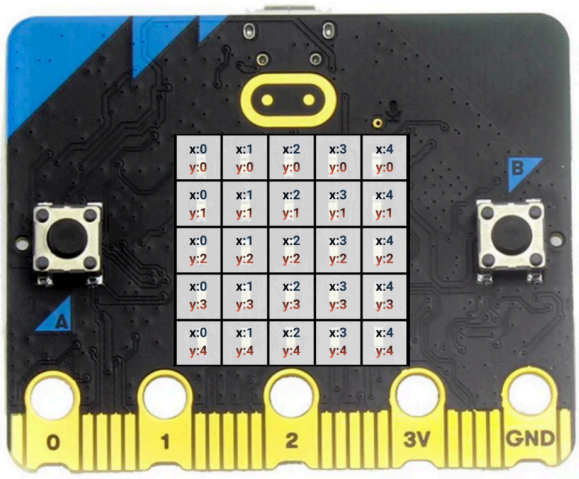


Рисунок 2

Если наш Корабль находится в левом нижнем углу, какие у него координаты?

(Ответ: X=0, Y=4).

> ## Интересный факт: Почему Y идет вниз?
>
> В математике ось Y всегда стремится вверх. Почему в компьютерах наоборот? Это наследие старых телевизоров и первых мониторов (ЭЛТ). Они рисовали изображение с помощью луча, который бежал по экрану, как мы читаем книгу: начинал в левом верхнем углу и двигался строчка за строчкой вниз. Поэтому для компьютера "начало мира" (0, 0) — это всегда левый верхний угол. Чем ниже луч опускался, тем больше становилась координата Y. Эта традиция сохранилась до сих пор во всех игровых движках!

Для управления экраном в MicroPython есть специальный модуль — **display**. Вы уже знакомы с ним по первому уроку. Представьте, что это ваш

личный художник, который ждет определенных команд.

В программировании все, чем мы управляем, называется **Объектами**.

- display — это объект (наш художник).
- gamepad — это объект (наш пульт).

Метод — это действие, которое объект умеет выполнять. Это команда для конкретного объекта.

Методы всегда пишутся через точку после имени объекта и обязательно со скобками ().

Пример из жизни:

- Объект: Собака
- Метод: Собака.лаять() или Собака.сидеть()

Пример из кода:

- Объект: display (Экран)
- Метод: display.clear() — Эй, экран, очистись!

Вот главные команды, будем далее называть их **методами**, которые нам понадобятся для создания игр:

1. **clear()** — Очистить экран.
   Выключает все светодиоды. Это как стереть всё с доски перед тем, как рисовать новый кадр мультика. Без этой команды старые точки останутся гореть, и вместо игры получится каша.
2. **set_pixel(x, y, яркость)** — Зажечь точку.
   Самая главная команда для геймдева. Мы говорим где (X, Y) и как ярко (от 0 до 9) зажечь свет.
   1. set_pixel(2, 2, 9) — зажечь центр на полную мощность.
   2. set_pixel(0, 0, 0) — выключить точку в углу.
3. **show(Image)** — Показать картинку.
   Выводит готовую иконку (например,

Image.HEART). Удобно для заставок или Game Over, но не подходит для самой игры, так как перерисовывает весь экран сразу.

Главное правило анимации:

Чтобы создать движение, нужно делать три шага в цикле:

1. **Стереть всё** (clear)
2. **Нарисовать объект на новом месте** (set_pixel)
3. **Подождать** (sleep), чтобы глаз успел заметить кадр.

Прежде чем начать, убедитесь, что в вашем проекте есть файл **gamepad.py** с прошлого урока!

Практическая работа №1: Рождение Героя

Наш корабль будет находиться на нижней линии (Y=4). Он может двигаться только влево и вправо (менять X).

В первую очередь подключим библиотеки:

```
from microbit import *
import gamepad
import random  # Подключаем генератор случайностей
```

> ### Справка: Что такое модуль?
> Модуль — это библиотека с дополнительными возможностями. Сам Python умеет многое (считать, сравнивать), но он не умеет всё. Чтобы не перегружать его, специальные функции (например, для математики, работы со временем или случайными числами) хранятся в отдельных "коробках" — модулях.

Чтобы воспользоваться ими, мы пишем уже знакомую вам команду **import**

Модуль random (англ. случайный) позволяет компьютеру «бросать кубик». Это делает игру интересной, ведь мы не знаем, где появится враг.

Главный метод, который нам нужен:

- **randint(a, b)** — выдать случайное целое число от A до B.

Теперь выделим с помощью комментария раздел параметров игры, здесь будут храниться созданные вами переменные, а также создадим первую переменную — **player_x**. — координата X игрока.

```
# --- НАСТРОЙКИ ИГРЫ ---
player_x = 2  # Игрок начинает в центре (0, 1, 2, 3, 4)
```

Далее инициализируем бесконечный цикл, а внутри запустим управление персонажем по джойстику:

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
    if gamepad.joystick_x() < 200 and player_x > 0:
        player_x -= 1
```

Рассмотрим строку условия:

Если мы наклоняем джойстик влево (значение меньше 200) **И** мы не у края (x > 0), то тогда игрок будет перемещен вправо – из текущего значения координаты вычитается единица.
Далее:

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
    if gamepad.joystick_x() < 200 and player_x > 0:
        player_x -= 1

    elif gamepad.joystick_x() > 800 and player_x < 4:
        player_x += 1
```

В альтернативном условии **elif** мы проверяем, если джойстик наклонен вправо (значение больше 800) **и** мы не у края (x < 4), то игрок будет перемещен вправо.

Далее переходим к отрисовке игрока:

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
    if gamepad.joystick_x() < 200 and player_x > 0:
        player_x -= 1

    # Если джойстик вправо (больше 800) И мы не у края (x
    elif gamepad.joystick_x() > 800 and player_x < 4:
        player_x += 1

    # 2. РИСОВАНИЕ
    display.clear()

    display.set_pixel(player_x, 4, 9)

    # 3. ПАУЗА (Скорость игры)
    sleep(150)
```

По уже изученной схеме с помощью методов clear() и set_pixel() производим отрисовку.

В конце обязательно добавляем паузу, чтобы операция выполнялась с видимым глазу интервалом.

Эксперимент: загрузите код. Попробуйте подвигать точкой с помощью джойстика!

> ***Обратите внимание**: Если бы мы не добавили проверки типа **and player_x > 0**, наша точка улетела бы за пределы экрана, и игра бы выдала ошибку!*

Практическая работа №2. Появление Врага

Враг — это тоже точка, но она живет своей жизнью.

1. Враг появляется сверху (Y=0) в случайном месте (X=Random).
2. Враг постоянно падает вниз (Y увеличивается).

Сейчас ваш скрипт выглядит следующим образом:

```
from microbit import *
import gamepad
import random # Подключаем генератор случайностей

# --- НАСТРОЙКИ ИГРЫ ---
player_x = 2  # Игрок начинает в центре (0, 1, 2, 3, 4)

# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
    if gamepad.joystick_x() < 200 and player_x > 0:
        player_x -= 1

    elif gamepad.joystick_x() > 800 and player_x < 4:
        player_x += 1

    # 2. РИСОВАНИЕ
    display.clear()

    display.set_pixel(player_x, 4, 9)
```

```
    # 3. ПАУЗА (Скорость игры)
    sleep(150)
```

Внесем дополнения для создания врагов. В первую очередь добавим переменные для координат врага, а также таймер появления врага.

```
# --- НАСТРОЙКИ ИГРЫ ---
player_x = 2  # Игрок начинает в центре (0, 1, 2, 3, 4)

enemy_x = random.randint(0, 4) # Случайный X от 0 до 4
enemy_y = 0                    # Враг сверху

timer = 0
```

Метод **random.randint(0, 4)** позволяет выбрать случайное число в заданном диапазоне.

Далее в бесконечный цикл добавим логику врага. **Важно!** Любые действия описываем до этапа отрисовки на матрице, чтобы программа не тратила дополнительно время, чтобы отрисовать сначала игрока, а потом врага.

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
…
    # 2. ЛОГИКА ВРАГА
    timer += 1
    if timer == 5: # Каждые 5 циклов враг делает шаг
        enemy_y += 1
        timer = 0 # Сбрасываем таймер
…
    # 3. РИСОВАНИЕ
…
```

В первую очередь настроим таймер, чтобы враги не пролетали весь экран за одну итерацию (повторение) бесконечного цикла.

Далее настроим появление нового врага, как только старый покинул карту:

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
…
    # 2. ЛОГИКА ВРАГА
    timer += 1
    if timer == 5: # Каждые 5 циклов враг делает шаг
        enemy_y += 1
        timer = 0 # Сбрасываем таймер

    # Если враг улетел за низ экрана - спавним нового
    if enemy_y > 4:
        enemy_x = random.randint(0, 4)
        enemy_y = 0
…
    # 3. РИСОВАНИЕ
…
```

Последним этапом в разделе рисование добавим отрисовку врага:

```
# --- ГЛАВНЫЙ ЦИКЛ ---
while True:
    # 1. ПРОВЕРКА УПРАВЛЕНИЯ
…
    # 2. ЛОГИКА ВРАГА
…
    # 3. РИСОВАНИЕ
    display.clear()

    display.set_pixel(player_x, 4, 9)
```

```
        display.set_pixel(enemy_x, enemy_y, 5) # Враг (Потуск
    …
```

Эксперимент! Проверьте, падает ли враг. Попробуйте увернуться от него.

Практическая работа №3. Лазерная пушка!

Теперь самое интересное. Мы добавим стрельбу.

Поскольку экран маленький (всего 5 пикселей в высоту), делать летящую пулю сложно для первого раза. Мы сделаем Лазер.

- Логика: Если нажата Красная кнопка (X) -> Проверяем, находится ли враг прямо над нами (**player_x == enemy_x**).
- Если Да -> Враг уничтожен (спавним нового).

```
# 3. СТРЕЛЬБА
if gamepad.button_X():
    for y in range(4):
        display.set_pixel(player_x, y, 3) # Тусклый луч
    sleep(50) # Короткая вспышка

    if player_x == enemy_x:
        gamepad.vibrate(100) # Вибрация от взрыва
        display.show(Image.TARGET) # Картинка взрыва
        sleep(200)

        # Создаем нового врага
        enemy_y = 0
        enemy_x = random.randint(0, 4)
```

Рассмотрим данную конструкцию: в первую очередь идет проверка нажатия кнопки —

gamepad.button_X(). Далее с помощью цикла **for** идет отрисовка луча от игрока вверх.

После чего идет проверка – если координата X игрока в момент нажатия кнопки совпадает с координатой X врага — **player_x == enemy_x**, то из нашего драйвера запускается метод **vibrate()**, вызывающий вибрацию геймпада, после чего происходит отрисовка картинки — **display.show(Image.TARGET)** и в конце создается новый враг.

Самостоятельная работа — челлендж: Game Over

Игра не имеет смысла, если в ней нельзя проиграть. Когда враг касается игрока — игра должна заканчиваться. Задание для самостоятельной работы:

Добавьте условие:

Если enemy_y равно 4 (низ экрана) И enemy_x равно player_x (координаты совпали):

1. Показать Image.SKULL (Череп).
2. Включить длинную вибрацию.
3. Остановить игру (можно использовать break для выхода из цикла или бесконечный sleep).

Ответ:

```
# 4. ПРОВЕРКА ПРОИГРЫША (GAME OVER)
# Если враг достиг низа (Y=4) И его X совпал с X игрока
if enemy_y == 4 and enemy_x == player_x:
    gamepad.vibrate(500)       # Длинная вибрация (жужжим
    display.show(Image.SKULL)  # Показываем череп
    break              # Прерывание основного цикла
```

> ## Это интересно! Баг, ставший фичей:
>
> В оригинальной Space Invaders 1978 года враги двигались медленно. Но по мере того, как игрок убивал их, их становилось меньше. Процессору становилось "легче" их отрисовывать, и игра начинала работать быстрее. Враги ускорялись не специально — это была ошибка (баг) слабого железа! Но разработчикам это так понравилось, что они решили оставить это как механику сложности.

Поздравляю, вы написали свой первый игровой движок!

- Мы использовали **переменные** (player_x, enemy_x) для хранения состояния мира.
- Мы применили **генератор случайностей** (random), чтобы игра была непредсказуемой.
- Мы создали **коллизию** (проверку столкновений): if player_x == enemy_x.

